Deep Sweep Page — Information Document

Page Overview

Purpose

The Deep Sweep page runs a **one-off broad query across multiple repositories in parallel**. It is the multi-repository counterpart to the Deep Dive page: users pick a set of repositories, enter keywords and (optionally) a date range, cap the per-repository result count, and optionally enable AI summarisation. The backend creates a `deep_sweep` execution, queries each repository in sequence, normalises and de-duplicates the combined result set (by DOI, and by title + first-author), generates a single Markdown report, and — if AI is enabled — produces a per-document summary that is embedded in the report.

Primary User Flows

1. **Configure and run.** Select repositories → enter keywords → optionally set date range, max-results cap, AI toggle, and per-execution AI overrides → click **Run Deep Sweep**.
2. **Load a saved configuration.** Use the `ConfigLoader` at the top of the form to reapply a previously saved `manual_sweep` configuration; the date range is intentionally excluded from saved configurations and must be entered fresh each run.
3. **Save the current form as a configuration.** Open the **Save Configuration** modal, give it a name, and save to the `manual_sweep` configuration set.
4. **Track and follow-up.** On submit, the page receives an `execution_id` and registers it with the shared `ExecutionContext`; live progress appears in the floating widget and on the Monitor page. When the run finishes, a result card with counts and a link to the report is shown.

Inputs and Outputs

Inputs (form state):

- `repositories` — list of repository slugs to query (required, ≥ 1).

- `keywords` — list of keyword tokens (required, ≥ 1); joined with spaces to form the underlying `query` string.
- `dateFrom` / `dateTo` — optional ISO date strings; sent as `null` when empty.
- `maxResults` — integer 10–500 (step 10, default 100), **per repository**.
- `aiEnabled` — boolean toggle.
- `aiOverrideLength` / `aiOverrideTone` / `aiOverrideModel` (IMPL-AI13) — optional per-execution overrides; only non-empty values are included in the request body.
- `ephemeralKeys` — in-memory map of `{credential_name: key_value}` for keyed repositories whose key is not stored in the keyring.

Outputs:

- `POST /api/search/sweep` returns `{ execution_id: number }`.
- A `deep_sweep` execution row is persisted by `SweepEngine.prepare_execution` (with `parameters` JSON, `start_time`, and `execution_type = "deep_sweep"`).
- On completion: a Markdown report under `resmon_reports/markdowns/` and a per-task log under `resmon_reports/logs/`.
- Live progress events on the execution's progress channel (`execution_start`, `repo_start`, `query_progress`, `repo_done`, `dedup_stats`, `report_saved`, `complete`, etc.).

Known Constraints or Permissions

- **Admission controller.** Before launching, the backend calls `_reject_if_at_manual_cap()`; if the concurrent-execution limit is reached, the request is rejected with **HTTP 429** and a `Retry-After: 5` header. The error message references **Settings** → **Advanced** where the cap is configurable.
- **Required keys.** Key-less repositories (e.g. arXiv, CrossRef, OpenAlex, bioRxiv, medRxiv) always work; the four repositories listed in `_REQUIRED_CREDENTIALS` (`core`, `ieee`, `nasa_ads`, `springer`) require a stored or ephemeral key. A missing key does not fail the sweep — the engine emits a `repo_skipped_missing_key` event, the task log records a warning, and that repository's query returns zero results.
- **At least one repository and one keyword** are required; client-side validation prevents submission otherwise.
- **Per-repository cap, not global.** `max_results` applies to each repository individually; the merged, de-duplicated result set may be smaller than `repositories.length × max_results`.

- **Save-as-configuration omits the date range** by design; the date range must be chosen fresh on every run.

Frontend

Route and Main Component

- Route: `/sweep` (mounted in `App.tsx`).
- Main component: `DeepSweepPage` in `resmon_scripts/frontend/src/pages/DeepSweepPage.tsx`.

Child Components and Hooks

Child components rendered inside the form card:

- `PageHelp` — collapsible on-page help with "When to use this page", "How to use it", and "Tips" sections.
- `ConfigLoader` (`configType="manual_sweep"`) — dropdown to load saved configurations; a `refreshKey` prop triggers a reload after a save.
- `RepositorySelector` (`mode="multi"`) — multi-select over the repository catalog.
- `RepoKeyStatus` (`variant="sweep"`) — rendered once per selected repository; shows whether the credential is stored, absent, or accepted as an ephemeral value, and offers inline entry for ephemeral keys.
- `DateRangePicker` — two date fields wired to `dateFrom` / `dateTo`.
- `KeywordInput` — chip-style keyword entry (Enter-to-commit).
- `InfoTooltip` — small contextual hints next to the max-results and AI toggles.

Hooks:

- `useExecution()` (from `ExecutionContext`) — provides `activeExecutions` and `startExecution(id, type, repositories)`.
- `useRepoCatalog()` — returns `bySlug`, `presence`, and `refreshPresence()` for catalog-driven key-presence indicators.
- `React.useRef<number | null>(null)` stored as `pageExecIdRef` to track the execution launched from this page instance.
- `React.useEffect` watching `pageExec?.status`: when the tracked execution leaves `running` / `cancelling`, `pageExecIdRef.current` is cleared so the page is ready for a new run.

UI State Model

Local state (`useState`):

State	Type	Purpose
`ephemeralKeys`	`Record<string, string>`	Per-credential-name ephemeral key buffer; only non-empty entries are sent on submit.
`repositories`	`string[]`	Selected repository slugs.
`dateFrom` , `dateTo`	`string`	Optional ISO date strings.
`keywords`	`string[]`	Keyword chips.
`maxResults`	`number`	Per-repository cap (10–500 step 10, default 100).
`aiEnabled`	`boolean`	AI summarisation toggle.
`aiOverrideLength` / `aiOverrideTone` / `aiOverrideModel`	`string`	IMPL-AI13 per-execution overrides; empty means "use app default".
`error`	`string`	Inline form error (repo/keyword validation or submit failure).
`saveModalOpen` , `configName` , `saveStatus`	—	State for the Save Configuration modal.
`configRefresh`	`number`	Bump counter that re-triggers `ConfigLoader` after a save.

Refs:

- `pageExecIdRef: React.RefObject<number | null>` — tracks the ID of the execution launched from this page so the result card and the "Run Another" button label reflect the correct run.

Derived values:

- `pageExec` — looked up as `activeExecutions[pageExecIdRef.current]` .
- `running` — true when `pageExec?.status === 'running'` .
- `buildQuery()` — `keywords.join(' ')` .

Key Interactions and Events

- **Form submit (`handleRun`).** Validates `repositories.length ≥ 1` and `keywords.length ≥ 1` ; assembles the request body; filters `ephemeralKeys` to non-empty values and sends them as `ephemeral_credentials` ; includes `ai_settings` only when at least one of `length` / `tone` / `model` overrides is non-empty; POSTs to `/api/search/sweep` ; on success, stores the returned `execution_id` in `pageExecIdRef` and calls `startExecution(id, 'deep_sweep', repositories)` so the shared execution context picks it up.
- **Config load (`applyConfig`).** Populates `repositories` , `keywords` , `maxResults` , and `aiEnabled` from the loaded configuration. `dateFrom` and `dateTo` are intentionally **not** restored.
- **Save Configuration (`handleSaveConfig`).** POSTs to `/api/configurations` with `config_type: "manual_sweep"` and a `parameters` payload containing `repositories` , `date_from` , `date_to` , `keywords` , `max_results` , and `ai_enabled` . On success, closes the modal, clears the name, bumps `configRefresh` , and shows a transient success message that clears after 3 seconds.
- **Ephemeral key entry.** `RepoKeyStatus` invokes `onEphemeralChange(v)` , which updates `ephemeralKeys[credential_name]` . `onPresenceChange` calls `refreshPresence()` from `useRepoCatalog` to re-fetch presence status.
- **Execution status transition.** The `useEffect` on `pageExec?.status` resets `pageExecIdRef` when the tracked execution leaves the running set, enabling a subsequent run from the same page without a reload.

Error and Empty States

- **Empty repositories selection.** Submit sets `error = 'Please select at least one repository.'` .
- **Empty keywords.** Submit sets `error = 'Please enter at least one keyword.'` .
- **Submit exception.** The `catch` block in `handleRun` sets `error` to the caught message (e.g. the 429 from the admission controller) or the generic fallback `'Failed to start sweep.'` .
- **Completed/failed result card.** When `pageExec` exists and its status is neither `running` nor `cancelling` , a `result-card` is rendered. The CSS class is `result-success` when `status === 'completed'` and `result-failure` otherwise; it shows the execution ID, status, result

count, new count, elapsed seconds (when available), and a **View Report** link pointing to `#/results?exec=<id>`.
- **No per-repository key status.** The Key Status panel is rendered only when `repositories.length > 0`.

Backend

API Endpoints

Endpoints used by this page:

- `POST /api/search/sweep` — launches a deep-sweep execution.
- `POST /api/configurations` — used by the Save Configuration modal (`config_type: "manual_sweep"`).
- `GET /api/repositories/catalog` and `GET /api/credentials` — consumed through `useRepoCatalog()` / `RepoKeyStatus` for presence state.

Request and Response Patterns

`POST /api/search/sweep` is handled by `search_sweep(body: SweepRequest)` in `resmon_scripts/resmon.py`. `SweepRequest` is the following Pydantic model:

```
class SweepRequest(BaseModel):
    repositories: list[str]
    query: str
    keywords: Optional[list[str]] = None
    date_from: Optional[str] = None
    date_to: Optional[str] = None
    max_results: int = 100
    ai_enabled: bool = False
    ai_settings: Optional[dict] = None
    ephemeral_credentials: Optional[dict[str, str]] = None
```

The handler:

1. Calls `_reject_if_at_manual_cap()`. If `admission.try_admit(kind="manual")` returns false, FastAPI raises `HTTPException(status_code=429, …, headers={"Retry-After": "5"})` with a detail message referencing **Settings** → **Advanced**.
2. Builds a `SweepEngine` with `config={"ai_enabled": body.ai_enabled, "ai_settings": body.ai_settings}`.

3. Calls `engine.prepare_execution("deep_sweep", body.repositories, query_params)` to insert the execution row and receive `exec_id`.
4. Calls `progress_store.register(exec_id)` and then `_launch_execution(engine, exec_id, conn, ephemeral_credentials=body.ephemeral_credentials)`.
5. Returns `{ "execution_id": exec_id }`.

Response shape: `{ execution_id: number }`. All further state arrives via progress events and the execution record.

`POST /api/configurations` accepts a `ConfigCreate` (`name`, `config_type`, `parameters`) and is used unchanged by the Save Configuration modal.

Persistence Touchpoints

- **`executions` table.** `SweepEngine.prepare_execution` inserts a row with `execution_type = "deep_sweep"`, a JSON-serialised `parameters` dict (which includes `repositories`), and `start_time`. The row is later updated by `update_execution_status` with `end_time`, `result_count`, `new_result_count`, `log_path`, `result_path`, and (on full failure) an `error_message`.
- **`documents` table.** For each raw result, `normalize_result` + `validate_result` + `get_document_by_source` / `link_execution_document` persists document rows and creates execution ↔ document associations. `first_seen_at` is compared against the execution's `start_time` to compute the `new_count`.
- **`configurations` table.** `POST /api/configurations` writes a row with `config_type = "manual_sweep"` and the JSON `parameters` payload.
- **`app_settings`.** Read in `_apply_ai_settings_to_engine` to obtain the persisted `ai_*` keys listed in `_AI_SETTING_KEYS`.
- **Progress events.** Emitted in memory during the run via `progress_store.emit`; `save_progress_events(conn, exec_id, events)` persists the event stream into the database at the end of `_launch_execution`.

Execution Side Effects

`_launch_execution` starts a daemon thread that runs the full pipeline; the HTTP request returns immediately. The thread:

1. **Admission bookkeeping** — `admission.note_admitted(exec_id)` on entry, `admission.note_finished(exec_id)` in the `finally` block.

2. **Ephemeral credential registration** — `push_ephemeral(exec_id, ephemeral_credentials or {})` so that `get_credential_for(exec_id, name)` used inside API clients sees the per-execution keys; `pop_ephemeral(exec_id)` runs in `finally`.
3. **AI settings resolution (`_apply_ai_settings_to_engine`).** Loads persisted `ai_*` settings and merges them with the per-execution override (`engine.config["ai_settings"]`), after translating IMPL-AI13 short keys through `_AI_OVERRIDE_KEY_MAP` (`length → ai_summary_length` , `tone → ai_tone` , `model → ai_model`). Builds `engine.config["ai_prompt_params"]` (length, tone, extraction_goals, `_show_audit_prefix` , `_audit_provider` , `_audit_model`). Calls `build_llm_client_from_settings` . If `ai_enabled` is true but the factory returns `None` or raises `ValueError` , a single `log_entry` warning progress event is emitted (`"AI skipped: …"`) and the execution continues with `engine.llm_client = None` .
4. **Pipeline execution (`SweepEngine.run_prepared(exec_id)`)** — see the stage list below.
5. **Progress finalisation.** `progress_store.mark_complete(exec_id)` → persist events via `save_progress_events` → `progress_store.cleanup(exec_id)` .
6. **Routine completion email hook (IMPL-R7).** Not fired for manual sweeps because the branch is gated on `execution_type == "automated_sweep"` and a non-null `routine_id` .

Sweep pipeline stages (inside `SweepEngine._run` , driven by `run_prepared`):

1. **`execution_start` event** with the repositories list and `total_repos` .
2. **`stage = "querying"`** — iterate `repositories` :
   - Emit `repo_start` ; consult `_REQUIRED_CREDENTIALS` and, if the repository needs a key that is not available, emit `repo_skipped_missing_key` and append to `missing_key_repos` (the query still runs, but the upstream client typically returns zero results).
   - Call `get_client(repo_name)` ; set `client._exec_id = exec_id` so the client can look up ephemeral credentials via `get_credential_for` .
   - Run `_search_with_heartbeat(...)` , which performs the HTTP query subject to each client's built-in rate limiting / retry logic and emits `heartbeat` events while the call is in flight.
   - Emit `query_progress` (`search_start` , `search_done`) and `repo_done` with the per-repo result count. Errors are caught,

logged, appended to `repo_errors`, and emitted as `repo_error` events; the sweep continues with the next repository.
- Cancellation is checked at every iteration via `progress_store.should_cancel(exec_id)`.

3. **`stage = "dedup"`** — `deduplicate_batch(self.db, all_results)` runs cross-source deduplication and returns counts (`total`, `new`, `duplicates`, `invalid`), emitted as `dedup_stats`.
4. **`stage = "linking"`** — for each raw result, normalise/validate, look up or create the `documents` row, and link it to the execution via `link_execution_document`. Progress is emitted as `link_progress` at ~5% granularity.
5. **`stage = "reporting"`** — `_build_report_docs(all_results)` assembles the per-document dicts used for the report.
6. **Optional `stage = "summarizing"` (only when `self.llm_client` is set, `ai_enabled` is true, and there are documents to summarise).** Iterates documents one at a time so cancellation and per-document progress can be honoured; `SummarizationPipeline.summarize_document(text)` is invoked per entry with the prompt params built earlier. Failures on a single document are logged and skipped (the batch continues); an `ai_summary` key is added to each successful document dict. Emits `ai_start`, `ai_progress` per document, and `ai_done`.
7. **Report generation.** `generate_report(report_docs, report_metadata)` produces Markdown including the query, keywords, repositories, `missing_key_repos`, date range, totals, and (when set) the `provider/model` audit label. The report is saved to `REPORTS_DIR/markdowns/report_deep_sweep_<exec_id>_<ts>.md`; a `report_saved` event is emitted.
8. **`stage = "finalizing"`** — `update_execution_status` writes final counts, `log_path`, `result_path`, and status. Status is `failed` only when *every* requested repository errored (`all_repos_errored`); otherwise it is `completed`, even if a subset failed (their errors remain in the task log and as `repo_error` events).
9. **Auto-backup.** `_maybe_auto_backup(...)` spawns a daemon thread to optionally push the report and log to cloud storage; failures are logged but never fail the execution.
10. **Final events.** `complete` event (with `status`, `result_count`, `new_count`, `elapsed`, and — on full failure — the `error` summary), followed by `progress_store.mark_complete(exec_id)`.

Cancellation. `_handle_cancellation` is invoked if the cancel flag is set at any inter-stage checkpoint or between documents in the AI step; it writes a partial report from `all_results` collected up to that point, updates the execution row to `cancelled`, and emits the terminal events.

Deep Sweep Page — Information Document

Page Overview

Purpose

Primary User Flows

Inputs and Outputs

Known Constraints or Permissions

Frontend

Route and Main Component

Child Components and Hooks

UI State Model

Key Interactions and Events

Error and Empty States

Backend

API Endpoints

Request and Response Patterns

Persistence Touchpoints

Execution Side Effects